Pass 2 — deferred drops (kit architecture not integrated)

The Pass 2 batch (M94–M103) was written by the external contributor against a "kit" architecture from Pass 1 (M83–M93) that was **never integrated into this app** — the contributor's own `PASS2RESULTS.md` calls that the defining finding of Pass 1 ("60,000 lines against a codebase it could not see; integrated none").

The following modules depend on kit primitives that **do not exist in this tree** (`ModeKit` , `SimLoop` , `Replay` , `FixedStep` , `HeadlessSim` , `Rng` , `DunkTiers` , `Legibility` , `Certification`), so they are kept here as reference rather than wired in — dropping them into the build would not compile, and the hard rule is to never replace a working core with a less-tested drop-in.

file	why deferred
`DunkSim.ts.reference` , `DunkMode.pass2.ts.reference`	M94 dunk migration onto `ModeKit` / `SimulatableMode` / `DunkTiers` . Our live `dunk` mode (M63 lineage, in the registry) is working and shipped; the kit it targets is absent. Would replace a working mode.
`dunk_migration_test.ts.reference`	92 tests, but import `Rng` / `HeadlessSim` / `DunkTiers` which do not exist here.
`verifySession.ts.reference` , `verify_session_test.ts.reference`	M102 server replay verification. Needs `Replay` + `HeadlessSim` + a client that sends a serialised replay + seed + hash. None of that pipeline exists; `POST /api/sessions` sends `{mode,score,won,duration}` only.
`CertificationPass2.ts.reference` , `certification_pass2_test.ts.reference`	M103 fleet scorecard. Imports `CriterionState` from M93 `Certification` , absent here. It is a CI record, not app runtime.

What WAS integrated from Pass 2 (self-contained, real value)

These carried no kit dependency and are live in the app + test corpus:

- **M95 canvas fix** — `app/game-surface.css` (imported in root layout) takes the portrait-phone game surface from ~33% to ~80% of screen; `lib/babylon/core/canvasFit.ts` caps DPR at 2 + a backing-pixel budget, wired into `ModeHarness` engine boot and resize. Tests: `scripts/canvas-fit-tests.ts` .

- **M96 groundGuard** — `lib/babylon/core/groundGuard.ts` + `scripts/ground-guard-tests.ts`. NOTE: our live `GroundRide.ts` **already zeroes vertical velocity on landing and hard-clamp** (lines 73/91), so M96's core "position corrected, velocity not" diagnosis does not match our code. The remaining improvement (re-seat to last-safe point instead of `hardFloorY=0` when the raycast genuinely misses) is available via `groundStep` but deferred pending on-device playtest of the three board modes — changing working board physics blind risks a regression.
- **M97 CameraFraming** — `lib/babylon/core/CameraFraming.ts` + `scripts/camera-framing-tests.ts`.
  Pure framing arithmetic; available for camera tuning.
- **M99 cameraPresets** — `lib/babylon/core/cameraPresets.ts` + `lib/babylon/config/cameraPresets.json`
- `scripts/camera-presets-tests.ts`. Measured radii + `planFor()`. Applying the radii to live cameras is deferred: the presets reference camera names (`nexus_cam_*`) and an ArcRotateCamera `lowerRadiusLimit` model that differ from our `CameraDirector` / `TargetCamera` setup, and the contributor's own note says "nothing here helps until skin weights are fixed" and it needs converging on a real device.
- **M100 scoreScale + captions** — `lib/babylon/core/scoreScale.ts`, `lib/babylon/core/captions.ts` (bus authored to the M82 contract), and `lib/babylon/ui/CaptionRegion.tsx` (standards-correct aria-live surface).
  Tests: `scripts/score-scale-tests.ts`. `CaptionRegion` compiles and is ready to mount, but has no cue producers wired yet (modes do not call `captions.cue`), so it is not force-mounted across the game components until producers exist — same principle as QuizCore added as a pure engine in M78.